TEMA1-10

Compendio Técnico: HTML5 y CSS3

1. Estructura y Semántica HTML

Estructura Base del Documento

Todo documento HTML5 debe seguir esta estructura mínima obligatoria:

HTML
```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8" />
   <title>Título de la página</title>
   </head>
<body>
   </body>
</html>
```

- `<!DOCTYPE html>`: Define que es HTML5.
- `<html lang="xx">`: Raíz del documento con atributo de idioma.
- `<head>`: Metadatos (no visibles), título y enlaces.
- `<body>`: Contenido visible.

Elementos Semánticos Estructurales (HTML5)

Estas etiquetas definen la estructura y significado del contenido, comportándose como elementos de bloque (`display: block`):

- `<header>`: Cabecera de la página o de una sección/artículo.
- `<nav>`: Menú de navegación principal.
- `<main>`: Contenido principal (único por página).
- `<section>`: Agrupación temática de contenido. Suele llevar un encabezado (`h2`-`h6`).
- `<article>`: Contenido independiente y autónomo (posts, noticias). Puede anidarse.
- `<aside>`: Contenido tangencial o relacionado indirectamente (barras laterales).
- `<footer>`: Pie de página (autor, copyright, contacto).

Otros Elementos Semánticos

- **`<details>` y `<summary>`**: Bloque desplegable. `<summary>` es el título visible cuando está contraído.
- **`<figure>` y `<figcaption>`**: Contenido independiente (imágenes, diagramas) con una leyenda opcional.
- **`<time>`**: Fecha/hora.

Comportamiento de Visualización

- **Bloque (Block):** Ocupan todo el ancho disponible y empiezan en nueva línea (ej. `<div>`, `<p>`, `<h1>`, `<section>`).
- **Línea (Inline):** Ocupan solo el espacio de su contenido (ej. `<span>`, `<a>`, `<img>`). No pueden contener elementos de bloque.

---

2. CSS: Sintaxis e Inclusión

Regla CSS

Estructura básica: `Selector { Propiedad: Valor; }`.

Métodos de Inclusión

Externo (Recomendado): Archivo `.css` vinculado mediante `<link>`.
HTML
<link rel="stylesheet" href="style.css" />

1. .
2. **Interno:** Dentro de `<style>` en el `<head>`.
3. **En línea:** Atributo `style` en la etiqueta (No recomendado).

---

3. Selectores CSS

Básicos

- **Universal:** `* { ... }` (Afecta a todo).
- **Etiqueta/Tipo:** `p { ... }`.
- **Clase:** `.nombreclase { ... }` (Atributo `class="..."`).
- **ID:** `#nombreid { ... }` (Atributo `id="..."`, único por página).
- **Descendente:** `div p { ... }` (Cualquier `p` dentro de `div`, sin importar el nivel).
- **Agrupación:** `h1, h2 { ... }` (Aplica a ambos).

Avanzados (Relacionales)

- **Hijo directo:** `p > span` (Solo hijos directos, no nietos).
- **Hermano adyacente:** `h1 + h2` (Elemento `h2` inmediatamente después de `h1`).
- **Hermano posterior (General):** `h2 ~ p` (Cualquier `p` que sea hermano y vaya después de `h2`).

De Atributo

- **`[attr]`**: Tiene el atributo.
- **`[attr="val"]`**: Valor exacto.
- **`[attr^="val"]`**: Comienza por "val".
- **`[attr$="val"]`**: Termina en "val".
- **`[attr*="val"]`**: Contiene "val".

---

4. Pseudoclases y Pseudoelementos

Pseudoclases (Estado y Estructura)

Se definen con dos puntos simples `:`.

- **Enlaces:** `:link` (no visitado), `:visited` (visitado).
- **Interacción:**
  - `:hover` (puntero encima).
  - `:active` (al hacer clic/activar).
  - `:focus` (tiene el foco, ej. inputs).
- **Estructurales:**
  - `:nth-child(n)`: Enésimo hijo (acepta `odd`, `even`, fórmulas `2n+1`).
  - `:first-child` / `:last-child`: Primer o último hijo.
  - `:only-child`: Único hijo.
  - `:empty`: Sin hijos ni texto.
  - `:not(selector)`: Negación (selecciona lo que no cumple el selector).
- **Navegación:** `:target` (elemento cuyo ID coincide con el ancla de la URL).

Pseudoelementos (Partes de un elemento)

Se recomienda la sintaxis de doble dos puntos `::`.

- **`::first-letter`**: Primera letra (ideal para letras capitulares).
- **`::first-line`**: Primera línea de texto.
- **`::selection`**: Texto seleccionado por el usuario.
- **`::placeholder`**: Texto de ayuda en inputs.

---

5. Unidades, Colores y Funciones

Unidades de Medida

- **Absolutas:** `px`, `pt`, `cm`. Poca flexibilidad.
- **Relativas (Recomendadas):**
  - **`em`**: Relativo al tamaño de fuente del elemento padre.
  - **`rem`**: Relativo al tamaño de fuente raíz (`html`), por defecto 16px.
  - **`%`**: Porcentaje respecto a una medida de referencia.
  - **Viewport:** `vw` (1% ancho ventana), `vh` (1% alto ventana).

Colores

- **Formatos:** Palabras clave (`red`), Hexadecimal (`#FF0000`, `#F00`), RGB (`rgb(255,0,0)`), HSL (`hsl(0, 100%, 50%)`).
- **Transparencia (Alpha):** `rgba(...)` y `hsla(...)`. El último valor es opacidad (0.0 a 1.0).
- **`currentColor`**: Variable que referencia el color actual del texto del elemento.

Función `calc()`

Permite realizar cálculos matemáticos mezclando unidades (`+`, `-`, `*`, `/`).

- Sintaxis: `width: calc(100% - 30px);`.

---

6. Variables CSS (Custom Properties)

Definición y Uso

Permiten valores reutilizables y modificables por JS.

Definición: Generalmente en `:root` (alcance global) usando doble guion `--`.
CSS
```
:root {
  --color-principal: #06c;
}
```

1. .

Uso: Mediante `var()`.
CSS
h1 { color: var(--color-principal); }

2. .
3. **Fallback:** `var(--nombre, valor_por_defecto)`.

---

7. Cascada, Especificidad y Herencia

Herencia

Ciertas propiedades (texto, color) se heredan de padres a hijos automáticamente. Se puede forzar con valores: `inherit` (forzar herencia), `initial` (valor por defecto), `unset` (resetear).

Resolución de Colisiones (Cascada)

Si hay reglas contradictorias, se aplican en este orden de prioridad (de menor a mayor):

1. Estilos del navegador (User Agent).
2. Estilos de usuario.
3. Estilos del autor (Diseñador).
4. `!important` (Rompe la cascada, usar con precaución).

Especificidad (Peso del selector)

Si la importancia es igual, gana el selector más específico. Cálculo del valor:

1. **Estilos en línea** (atributo `style`).
2. **ID** (`#`).
3. **Clases, atributos y pseudoclases** (`.`, `:hover`, `[type]`).
4. **Elementos y pseudoelementos** (`p`, `::before`).

Ejemplo: `#id` (0,1,0,0) gana a `.clase` (0,0,1,0).

Orden

A igual especificidad, prevalece la **última regla definida** en el código o el último archivo CSS vinculado.

Sugerencias Prácticas Basadas en los PDF

1. **Optimización de Selectores:**
    - Evita el uso excesivo de `!important`; en su lugar, aumenta la especificidad del selector o revisa el orden de carga de tus archivos CSS.
    - Aprovecha los selectores de atributo como `a[href^="http"]` para estilar enlaces externos de forma distinta a los internos.
2. **Mantenibilidad con Variables:**
    - Define colores, fuentes y espaciados base en `:root`. Esto permite cambiar el tema de toda la web modificando solo unas pocas líneas.
    - Usa variables sin unidades (ej. `--tamano: 10`) si necesitas multiplicarlas dentro de un `calc()`, ya que `calc()` no permite división por números con unidades.
3. **Estructura Semántica:**
    - No uses `<div>` para todo. Utiliza `<nav>`, `<article>` y `<section>` para mejorar la accesibilidad y el SEO. Recuerda que los navegadores asignan `display: block` a estas etiquetas por defecto, pero no estilos visuales.
    - Usa `<label>` y placeholders adecuados en formularios para mejorar la experiencia de usuario, y estiliza el placeholder usando `::placeholder`.
4. **Diseño Responsivo y Unidades:**
    - Prioriza unidades relativas (`em`, `rem`, `%`) sobre absolutas (`px`) para textos y contenedores principales. Esto facilita la adaptación a diferentes dispositivos.
    - Ten cuidado al usar `em` anidados, ya que el tamaño se multiplica (efecto bola de nieve). `rem` es más seguro al basarse siempre en el `html` raíz.
5. **Orden de Pseudoclases en Enlaces:**
    - Respeta estrictamente el orden **LVHA** para evitar conflictos de especificidad en enlaces: `:link`, `:visited`, `:hover`, `:active`.

TEMA 10-20

Compendio Técnico de CSS: Propiedades y Comportamiento

1. Modelo de Cajas (Box Model)

Todo elemento en CSS se representa como una caja rectangular compuesta por cuatro partes.

Estructura

1. **Contenido (content):** Texto, imágenes, etc..
2. **Relleno (padding):** Espacio transparente entre el contenido y el borde. Muestra el fondo de la caja.

3. **Borde (border):** Línea que encierra contenido y relleno.
4. **Margen (margin):** Separación transparente con cajas adyacentes. Muestra el fondo del padre.

Cálculo de Dimensiones (`box-sizing`)

Controla cómo se calculan `width` y `height`.

- **`content-box` (Valor inicial):** `width` y `height` solo afectan al contenido.
  - Ancho real = `width` + `padding` + `border`.
- **`border-box`:** `width` y `height` incluyen contenido, relleno y borde.
  - Ancho contenido = `width` - `padding` - `border`.

Propiedades de Dimensiones

- **`width` / `height`:** No admiten valores negativos. No aplican a elementos en línea (salvo imágenes).
- **`min-width` / `max-width`:** Establecen límites de anchura. Tienen prioridad sobre `width` fijo.
- **`min-height` / `max-height`:** Límites de altura.

Desbordamiento (`overflow`)

Controla qué pasa cuando el contenido no cabe en la caja.

- `visible`: El contenido sobresale (defecto).
- `hidden`: Se oculta lo que sobra.
- `scroll`: Muestra barras de desplazamiento siempre.
- `auto`: Muestra barras solo si son necesarias.

---

2. Márgenes y Relleno

Margen (`margin`)

Acepta valores negativos y la palabra clave `auto`.

- **Shorthand:** `margin: top right bottom left` (sentido agujas del reloj).
- **Centrado horizontal:** Se logra con una anchura definida y `margin-left: auto; margin-right: auto;`.
- **Colapso de márgenes verticales (Fusión):**
  - Entre hermanos: Se fusionan y queda el de mayor tamaño. Si son negativos, queda el de mayor magnitud (el más negativo).

    - Padre e hijo: Los márgenes verticales se fusionan si no hay borde o padding intermedio en el padre.

Relleno (`padding`)

Espacio interior. No admite valores negativos ni `auto`. La sintaxis shorthand es idéntica a `margin`.

---

3. Bordes y Contornos

Bordes (`border`)

Propiedad shorthand para anchura, estilo y color (`border: 1px solid red`).

- **Estilo (`border-style`):** `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`. Si es `none`, no se muestra.
- **Radio (`border-radius`):** Redondea esquinas. Admite hasta 8 valores (radio horizontal / vertical). `border-radius: 50%` crea un círculo si la caja es cuadrada.

Imágenes en Bordes (`border-image`)

Permite usar una imagen en lugar del estilo normal.

- **`border-image-slice`:** Define cómo recortar la imagen en 9 regiones. El valor `fill` preserva el centro.
- **`border-image-repeat`:** `stretch` (estirar), `repeat` (repetir), `round` (mosaico reescalado), `space` (mosaico con espacios).

Contorno (`outline`)

Línea exterior al borde.

- **Diferencias con border:** No ocupa espacio (no afecta al modelo de caja), no permite redondear esquinas, es igual en los 4 lados.
- **`outline-offset`:** Separa el contorno del borde (admite negativos para dibujarlo dentro).

---

4. Fondos (`background`)

Propiedad shorthand para definir color e imagen.

- **`background-image`:** `url('ruta')`. Se pueden apilar varias separadas por comas.
- **`background-repeat`:** `repeat`, `no-repeat`, `repeat-x`, `repeat-y`, `space`, `round`.
- **`background-position`:** Porcentajes (x% y%) o palabras clave (`center`, `top`, etc.). Origen: esquina superior izquierda.
- **`background-size`:**
  - `contain`: Escala para ver toda la imagen (pueden sobrar espacios).
  - `cover`: Escala para cubrir toda la caja (puede recortar imagen).
- **`background-attachment`:**
  - `scroll`: Se mueve con el elemento (fijo al borde).
  - `fixed`: Fijo respecto al viewport.
  - `local`: Se mueve con el contenido del elemento (scroll interno).
- **`background-clip`:** Define hasta dónde se pinta el fondo (`border-box`, `padding-box`, `content-box`).
- **`background-blend-mode`:** Mezcla capas de fondo (`multiply`, `screen`, `overlay`, etc.).

---

5. Texto y Tipografía

Fuentes

- **`font-family`:** Lista de fuentes ordenadas por preferencia, terminando en familia genérica (`serif`, `sans-serif`, `monospace`).
- **`font-size`:** Unidades: `px` (absoluta), `em`/`%` (relativa al padre), `rem` (relativa a `html`). Se recomienda `html { font-size: 62.5% }` para que 1rem = 10px.
- **`font-weight`:** Grosor (`normal`=400, `bold`=700).
- **`font-style`:** `italic` (cursiva), `oblique`.

Formato de Párrafo

- **`text-align`:** `left`, `right`, `center`, `justify`.
- **`line-height`:** Altura de línea. Usar un número sin unidad multiplica el tamaño de fuente. Igualar `line-height` a `height` centra el texto verticalmente en una sola línea.
- **`text-indent`:** Sangría de la primera línea.
- **`text-decoration`:** `none` (quita subrayado), `underline`, `line-through`, `overline`.
- **`text-transform`:** `uppercase`, `lowercase`, `capitalize`.
- **`letter-spacing` / `word-spacing`:** Espacio entre letras y palabras.

Sombras y Efectos de Texto

- **`text-shadow`**: `x y blur color`.
- **`white-space`**: Controla espacios y saltos. `pre` (respeta todo), `nowrap` (no salta línea), `pre-wrap` (respeta y ajusta).
- **`text-overflow`**: `ellipsis` añade "..." cuando el texto se recorta (requiere `overflow: hidden` y `white-space: nowrap`).

---

6. Enlaces y URL

Estructura URL

- **Absoluta:** Protocolo + Servidor + Ruta.
- **Relativa:**
  - `archivo.html` (mismo directorio).
  - `../archivo.html` (directorio superior).
  - `/ruta/archivo.html` (desde la raíz del servidor).
- **Anclas:** `#id` permite saltar a una posición específica del documento.

Etiqueta `<a>`

- `href`: Destino.
- `target="_blank"`: Abre en nueva pestaña.
- `download`: Fuerza descarga en lugar de visualización.

---

7. Transformaciones (2D y 3D)

Propiedad `transform`. No afecta al flujo normal (no mueve otras cajas).

Funciones 2D

- `translate(x, y)`: Mueve el elemento.
- `scale(x, y)`: Cambia tamaño (1 = original).
- `rotate(deg)`: Gira (sentido agujas del reloj).
- `skew(x, y)`: Inclina/deforma.

Funciones 3D

- Requiere definir `perspective` en el padre para efecto visual.
- `translateZ(z)`, `rotateX(deg)`, `rotateY(deg)`.

---

8. Visibilidad, Opacidad y Recorte

Ocultar Elementos

1. **`display: none`:** Elimina la caja del flujo (el espacio se ocupa por otros).
2. **`visibility: hidden`:** Oculta la caja pero mantiene su espacio reservado.
3. **`opacity: 0`:** Totalmente transparente, ocupa espacio. Animable.

Sombras de Caja (`box-shadow`)

`h-offset v-offset blur spread color inset`.

- A diferencia del contorno, sigue el `border-radius`.
- `inset`: Sombra interior.

Recorte (`clip-path`)

Define un área visible; lo de fuera se oculta.

- Formas: `circle()`, `ellipse()`, `inset()` (rectángulo), `polygon()`.
- **Nota:** `box-shadow` y `outline` son recortados y no se ven fuera del área de recorte.

---

9. Sugerencias

- **Evita cálculos manuales:** Usa `box-sizing: border-box` en el selector universal `*` para que el padding y border no aumenten el tamaño total de las cajas.
- **Centrado vertical de texto:** Para una sola línea de texto, establece `line-height` igual a la altura (`height`) del contenedor.
- **Márgenes verticales:** Evita definir `margin-top` y `margin-bottom` simultáneamente para prevenir problemas de colapso/fusión. Se recomienda usar solo `margin-bottom` para separar elementos.
- **Imágenes de fondo responsive:** Usa `background-size: cover` para que la imagen cubra siempre el contenedor sin deformarse, ideal para fondos de pantalla completa.
- **Texto largo:** Si necesitas puntos suspensivos (...) en textos largos, usa la combinación: `white-space: nowrap`, `overflow: hidden` y `text-overflow: ellipsis`.
- **Optimización de selectores:** Al usar fuentes web (Google Fonts), incluye la etiqueta `<link>` antes de tus propios estilos CSS.

- **Seguridad en enlaces:** Al usar enlaces con `target="_blank"`, ten en cuenta que abren un nuevo contexto de navegación.

TEMA 20-30

Compendio Técnico: Maquetación Avanzada y Multimedia

1. Modelos de Formato Visual (`display`)

Define el tipo de caja generada por un elemento.

- **`display: block`**: Ocupa todo el ancho, fuerza salto de línea.
- **`display: inline`**: Ocupa solo su contenido, no salta línea. No admite `width`, `height`, ni márgenes/padding verticales (visuales sí, pero no desplazan).
- **`display: inline-block`**: En línea por fuera, bloque por dentro (admite dimensiones).
- **`display: none`**: El elemento desaparece del flujo (no ocupa espacio).
- **`display: table`**: Comportamiento de tabla (`table-row`, `table-cell`, etc.).

Alineación Vertical (`vertical-align`)

Solo afecta a elementos `inline`, `inline-block` y celdas de tabla.

- **Valores:** `baseline` (defecto), `top`, `middle`, `bottom`, `sub`, `super`.

---

2. Posicionamiento (`position`)

Propiedad `position` combinada con coordenadas `top`, `right`, `bottom`, `left` y profundidad `z-index`.

- **`static`**: Flujo normal. Ignora coordenadas y z-index.
- **`relative`**: Se desplaza respecto a su posición original. **No** libera su espacio original en el flujo.
- **`absolute`**: Sale del flujo normal (no ocupa espacio). Se posiciona respecto al **primer ancestro posicionado** (distinto de static). Si no hay, respecto al viewport.

    - *Dimensiones:* Si no tiene ancho definido, se ajusta al contenido.
- **`fixed`**: Sale del flujo. Se posiciona respecto al **viewport** (ventana del navegador). No se mueve al hacer scroll.
- **`sticky`**: Híbrido. Se comporta como `relative` hasta un umbral de desplazamiento (scroll), luego se fija como `fixed`.

Apilamiento (`z-index`)

Controla el eje Z. Solo funciona en elementos posicionados (no static).

- **Valores:** Enteros (+ o -). Mayor número = más cerca del usuario.
- **Contexto:** Un hijo nunca puede estar por encima del padre si el padre tiene un z-index menor que otro elemento hermano.

---

3. Flexbox (Cajas Flexibles)

Modelo unidimensional (fila o columna).

Propiedades del Contenedor (Padre)

- **`display: flex | inline-flex`**.
- **`flex-direction`**: `row` (defecto), `row-reverse`, `column`, `column-reverse`. Define el Eje Principal.
- **`flex-wrap`**: `nowrap` (defecto), `wrap` (salta línea), `wrap-reverse`.
- **`flex-flow`**: Shorthand de direction y wrap.
- **`justify-content`** (Eje Principal): `flex-start`, `flex-end`, `center`, `space-between` (extremos pegados), `space-around` (espacio alrededor).
- **`align-items`** (Eje Secundario - por línea): `stretch` (defecto), `flex-start`, `flex-end`, `center`, `baseline`.
- **`align-content`** (Eje Secundario - multilínea): Distribuye el espacio entre líneas si hay `wrap`. Valores iguales a `justify-content` más `stretch`.

Propiedades de los Ítems (Hijos)

- **`order`**: Entero. Define el orden visual (defecto 0).
- **`flex-grow`**: Factor de crecimiento para ocupar espacio sobrante (defecto 0).
- **`flex-shrink`**: Factor de encogimiento si falta espacio (defecto 1).
- **`flex-basis`**: Tamaño inicial antes de distribuir espacio (`auto` o medida).
- **`flex`**: Shorthand (`flex-grow flex-shrink flex-basis`). Recomendado `flex: 1` (1 1 0%).
- **`align-self`**: Sobrescribe `align-items` para un hijo individual.

---

4. CSS Grid Layout (Cuadrícula)

Modelo bidimensional (filas y columnas).

Propiedades del Contenedor

- **`display: grid | inline-grid`**.
- **Definición de pistas:** `grid-template-columns` y `grid-template-rows`.
  - Unidades: `px`, `%`, `fr` (fracción del espacio disponible), `auto`.
  - Funciones: `repeat(n, tamaño)`, `minmax(min, max)`.
  - Palabras clave: `auto-fill` (llena con columnas), `auto-fit` (ajusta al contenido).
- **`grid-template-areas`**: Define el layout usando nombres de áreas entre comillas.
- **Huecos (Gutters):** `grid-column-gap`, `grid-row-gap`, `grid-gap` (shorthand).
- **Alineación (Similar a Flexbox):**
  - `justify-items` / `align-items`: Alinea contenido dentro de la celda.
  - `justify-content` / `align-content`: Alinea toda la rejilla dentro del contenedor.
- **Grid Implícito:** `grid-auto-rows` / `grid-auto-columns` y `grid-auto-flow` (controla si se añaden filas o columnas automáticamente).

Propiedades de los Ítems

- **Ubicación:** `grid-column-start`, `grid-column-end`, `grid-row-start`, `grid-row-end`.
  - Shorthand: `grid-column: inicio / fin` (admite `span n` para expandirse).
- **`grid-area`**: Asigna el ítem a un nombre de área definido en el padre o shorthand de posición (`fila-ini / col-ini / fila-fin / col-fin`).
- **Alineación propia:** `justify-self`, `align-self`.

---

5. Flotantes (`float`)

Saca el elemento del flujo normal y lo desplaza a izquierda o derecha.

- **Valores:** `left`, `right`, `none`.
- **Comportamiento:** Texto y elementos en línea fluyen alrededor. Elementos de bloque lo ignoran (se solapan) pero su contenido de texto sí se adapta.
- **`clear`**: `left`, `right`, `both`. Evita que un elemento tenga flotantes adyacentes.
- **Problema del contenedor (colapso de altura):** Si todos los hijos flotan, el padre tiene altura 0.

    - *Soluciones:* `overflow: hidden` en el padre o técnica "Clearfix" (pseudoelemento `::after { content: ""; clear: both; display: table; }`).
- **`shape-outside`**: Permite que el texto fluya alrededor de una forma no rectangular (círculo, polígono, imagen con transparencia). Requiere que el elemento esté flotando.

---

6. Imágenes y Multimedia

Etiqueta `<img>`

- Atributos: `src` (URL), `alt` (obligatorio), `width/height`.
- **`srcset`**: Lista de imágenes con su ancho (`200w`) o densidad (`2x`) para que el navegador elija la óptima.
- **`sizes`**: Define el tamaño que ocupará la imagen en el layout según media queries, ayudando a elegir el `srcset`.

Etiqueta `<picture>`

Permite dirección de arte (cambiar la imagen totalmente según media queries). Usa etiquetas `<source>` internas y un `<img>` final.

Propiedades CSS para Multimedia

- **`object-fit`**: Controla cómo el contenido reemplazado (img, video) se ajusta a su caja.
    - `fill` (estira, deforma), `contain` (entero, sobran espacios), `cover` (llena, recorta), `none`, `scale-down`.
- **`object-position`**: Alineación de la imagen dentro de la caja (ej. `50% 50%`).
- **`filter`**: Efectos visuales (`blur`, `brightness`, `contrast`, `grayscale`, `sepia`, `drop-shadow`).

Recorte (`clip-path`)

Recorta la visibilidad del elemento (todo tipo de cajas).

- Formas: `circle()`, `ellipse()`, `inset()` (rectángulo), `polygon()`, `url(#id_svg)`.

---

7. Tablas

Estructura HTML

`<table>`, `<caption>` (título), `<thead>`, `<tbody>`, `<tfoot>`, `<tr>` (fila), `<th>` (cabecera), `td` (dato).

- Atributos de fusión: `rowspan="n"`, `colspan="n"`.

Propiedades CSS

- **`border-collapse`**: `collapse` (bordes compartidos) vs `separate` (bordes separados).
- **`border-spacing`**: Espacio entre celdas (solo en modelo `separate`).
- **`table-layout`**:
  - `auto`: Ancho basado en contenido.
  - `fixed`: Ancho basado en la primera fila y ancho de tabla (más rápido).
- **`caption-side`**: `top` o `bottom`.
- **`empty-cells`**: `show` o `hide`.

---

8. Listas

- **`list-style-type`**: `disc`, `circle`, `square`, `decimal`, `lower-roman`, `none` (para quitar viñetas).
- **`list-style-position`**: `outside` (defecto), `inside` (viñeta dentro de la caja de texto).
- **`list-style-image`**: `url('imagen.png')`.

---

9. Generación de Contenido (`::before` y `::after`)

Pseudoelementos para insertar contenido estético desde CSS. Requieren la propiedad `content`.

- **`content`**: Texto (`"Hola"`), URL imagen, contadores (`counter()`), atributos (`attr(href)`), comillas (`open-quote`).
- **Contadores:**
  - `counter-reset: nombre 0`: Inicializa.
  - `counter-increment: nombre 1`: Aumenta.
  - `content: counter(nombre)`: Muestra.

---

10. Transformaciones

Propiedad `transform`. No afecta al flujo (otras cajas no se mueven).

- **2D:** `translate(x,y)`, `scale(x,y)`, `rotate(g)`, `skew(x,y)`.
- **3D:** `translateZ`, `rotateX`, `rotateY`, `perspective` (necesaria en el padre para ver profundidad).
- **`transform-origin`**: Punto de eje de la transformación (defecto: centro).

Sugerencias Prácticas Basadas en los PDF

1. **Layout Moderno:** Prioriza **Grid** para estructuras generales de página (2D) y **Flexbox** para componentes alineados en una dimensión (menús, barras). Evita usar `float` para maquetar estructuras completas; úsalo solo para texto alrededor de imágenes.
2. **Centrado:**
   - Para centrado absoluto (vertical y horizontal), usa Flexbox (`justify-content: center; align-items: center;`) o Grid (`place-items: center;`).
   - El antiguo método de `position: absolute` + márgenes negativos es menos flexible.
3. **Imágenes Responsivas:** Usa siempre `max-width: 100%; height: auto;` en imágenes para evitar desbordamientos. Para optimización de carga, implementa `srcset` y `sizes`.
4. **Tablas:** Usa `table-layout: fixed` para mejorar el rendimiento de renderizado en tablas grandes y asegurar anchos de columna predecibles.
5. **Accesibilidad en Iconos:** Si usas fuentes icónicas o SVG para botones/enlaces, asegúrate de tener texto alternativo o atributos `aria-label`, ya que `content` en CSS a veces no es leído por lectores de pantalla.
6. **Z-Index:** Recuerda que `z-index` solo funciona si el elemento tiene una `position` distinta de `static` (o es un hijo flex/grid). Si algo no se superpone, revisa el posicionamiento del padre (contexto de apilamiento).

TEMA 30-35

Compendio Técnico: CSS Avanzado y Formularios HTML

1. Gradientes CSS

Funciones para generar imágenes de fondo con transiciones de color. No se pueden animar directamente.

Lineales (`linear-gradient`)

Transición en línea recta.

- **Sintaxis:** `linear-gradient(dirección, color1 [parada], color2 [parada], ...)`
- **Dirección:**
  - Palabras clave: `to right`, `to bottom`, `to top left` (esquina opuesta).
  - Grados: `45deg`, `-90deg`.
- **Paradas de color:** Porcentaje o longitud donde el color es puro. Si no se indica, se distribuyen equitativamente.
  - Ejemplo: `linear-gradient(to right, red, blue 50%)` (el azul empieza puro al 50% del ancho).
- **Repetición:** `repeating-linear-gradient(...)` repite el patrón indefinidamente.

Radiales (`radial-gradient`)

Transición desde un punto central hacia afuera.

- **Sintaxis:** `radial-gradient(forma tamaño at posición, color1, color2, ...)`
- **Forma:** `circle` o `ellipse`.
- **Tamaño:**
  - `closest-side`: Hasta el lado más cercano.
  - `farthest-corner` (defecto): Hasta la esquina más lejana.
- **Posición:** Coordenadas (`center`, `top left`, `50% 50%`).
- **Repetición:** `repeating-radial-gradient(...)`.

---

2. Animaciones y Transiciones

Transiciones (`transition`)

Suavizan el cambio de valor de una propiedad ante un evento (ej. `:hover`).

- **`transition-property`**: Nombre de la propiedad (`width`, `background-color`) o `all`.

- **`transition-duration`**: Tiempo (`0.5s`, `300ms`).
- **`transition-timing-function`**: Curva de velocidad.
    - `linear` (constante), `ease` (lento-rápido-lento), `ease-in` (entra lento), `ease-out` (sale lento), `cubic-bezier(n,n,n,n)`.
- **`transition-delay`**: Retardo antes de empezar.
- **Shorthand:** `transition: property duration timing-function delay`.

Animaciones (`animation`)

Secuencias complejas definidas con fotogramas clave (`@keyframes`).

Definición:
CSS
```
@keyframes nombre {
  from { ... } /* 0% */
  50% { ... }
  to { ... }   /* 100% */
}
```

-
- **Propiedades:**
    - `animation-name`: Nombre del keyframe.
    - `animation-duration`: Tiempo de un ciclo.
    - `animation-iteration-count`: Número de veces (`infinite` para bucle).
    - `animation-direction`: `normal`, `reverse`, `alternate` (ida y vuelta).
    - `animation-fill-mode`: Estado al finalizar (`forwards` mantiene el último fotograma, `backwards` aplica el primero antes de empezar).
    - `animation-play-state`: `running` o `paused`.
- **Shorthand:** `animation: name duration timing-function delay iteration-count direction fill-mode`.

---

3. Media Queries

Reglas CSS condicionales para diseño responsivo.

- **Sintaxis:** `@media tipo_medio and (condición) { ... }`
- **Tipos:** `screen`, `print`, `all`.
- **Condiciones comunes:**
    - `(min-width: 768px)`: Aplica si el ancho es >= 768px.
    - `(max-width: 1024px)`: Aplica si el ancho es <= 1024px.
    - `(orientation: landscape)`: Dispositivo apaisado.
- **Estrategias:**

- **Mobile-first:** Estilos base para móvil -> `@media (min-width: ...)` para tablet/escritorio.
- **Desktop-first:** Estilos base escritorio -> `@media (max-width: ...)` para pantallas menores.

---

4. Formularios HTML5

Estructura Base

- **`<form>`**: Contenedor.
  - `action`: URL de destino.
  - `method`: `GET` (datos en URL, visible) o `POST` (datos en cuerpo, oculto, ficheros).
  - `enctype="multipart/form-data"`: Obligatorio para subir ficheros (`type="file"`).
- **`<label>`**: Etiqueta de texto. Atributo `for="id_input"` vincula el texto al campo (clic en texto activa input).

Tipos de Entrada (`<input>`)

Atributo `type="..."`:

- **Texto:** `text`, `password`, `email` (valida formato), `tel` (teclado numérico en móviles), `url`, `search`.
- **Numérico:** `number` (con flechas), `range` (slider). Atributos: `min`, `max`, `step`.
- **Selección:**
  - `checkbox`: Casillas independientes (mismo `name` envía múltiples valores si se marcan varios). Atributo `checked`.
  - `radio`: Excluyentes (mismo `name` agrupa opciones, solo una seleccionable). Atributo `checked`.
- **Especiales:**
  - `color`: Selector de color (hexadecimal).
  - `file`: Subida de archivos (`multiple`, `accept`).
  - `hidden`: Dato oculto, no visible pero se envía.
  - `date`, `time`, `month`, `week`: Selectores de fecha/hora.

Otros Elementos de Formulario

- **`<textarea>`**: Texto multilínea. Atributos `rows`, `cols`. No es autoconclusa.
- **`<select>`**: Lista desplegable.
  - `<option value="...">`: Cada opción.
  - `<optgroup label="...">`: Agrupa opciones.

    - Atributo `multiple`: Permite selección múltiple.
- **`<button>`**: Más flexible que `input type="submit"`. Admite HTML dentro (iconos). Tipos: `submit`, `reset`, `button`.
- **`<datalist>`**: Autocompletado para un `input`. Se vincula con `input list="id_datalist"`.

Atributos de Validación y Estado

- `required`: Campo obligatorio.
- `placeholder`: Texto de ayuda (desaparece al escribir).
- `disabled`: Desactivado (no se envía).
- `readonly`: Solo lectura (se envía).
- `pattern`: Expresión regular para validar (`[0-9]{9}`).
- `autofocus`: Foco automático al cargar.

Pseudoclases de Formulario (CSS)

- `:checked`: Elemento seleccionado.
- `:valid` / `:invalid`: Según validación HTML5.
- `:required` / `:optional`: Según atributo required.
- `:focus`: Tiene el foco.
- `:disabled` / `:enabled`.

---

5. Elementos Multimedia

Audio (`<audio>`) y Video (`<video>`)

- **Atributos comunes:**
    - `src`: Ruta del archivo.
    - `controls`: Muestra interfaz (play, volumen).
    - `autoplay`: Reproducción automática (restringido en móviles/algunos navegadores).
    - `loop`: Bucle.
    - `preload`: `auto`, `metadata`, `none`.

Formatos múltiples: Usar etiquetas `<source>` dentro para compatibilidad.
HTML
```
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.ogv" type="video/ogg">
</video>
```

-
- **Video específico:** `poster="imagen.jpg"` (miniatura inicial).

Sugerencias Prácticas Basadas en los PDF

1. **Validación Visual:** Usa las pseudoclases `:valid` e `:invalid` combinadas con `box-shadow` o `border` para dar feedback inmediato al usuario sobre el estado de los campos del formulario.
2. **Accesibilidad en Formularios:** Siempre usa `<label for="...">` o envuelve el input con el label. Esto aumenta el área de clic y es vital para lectores de pantalla.
3. **Optimización de Animaciones:** Anima preferentemente propiedades como `transform` y `opacity` en lugar de `width`, `height` o `top/left`, para mejor rendimiento (aceleración por hardware).
4. **Patrones de Fondo:** Usa `repeating-linear-gradient` con paradas de color bruscas (mismo % de fin de un color e inicio del siguiente) para crear patrones de rayas o cuadros sin imágenes.
5. **Media Queries Escalables:** Usa unidades relativas (`em` o `rem`) en los breakpoints de las media queries en lugar de `px` para que el diseño respete la configuración de tamaño de fuente del usuario.